



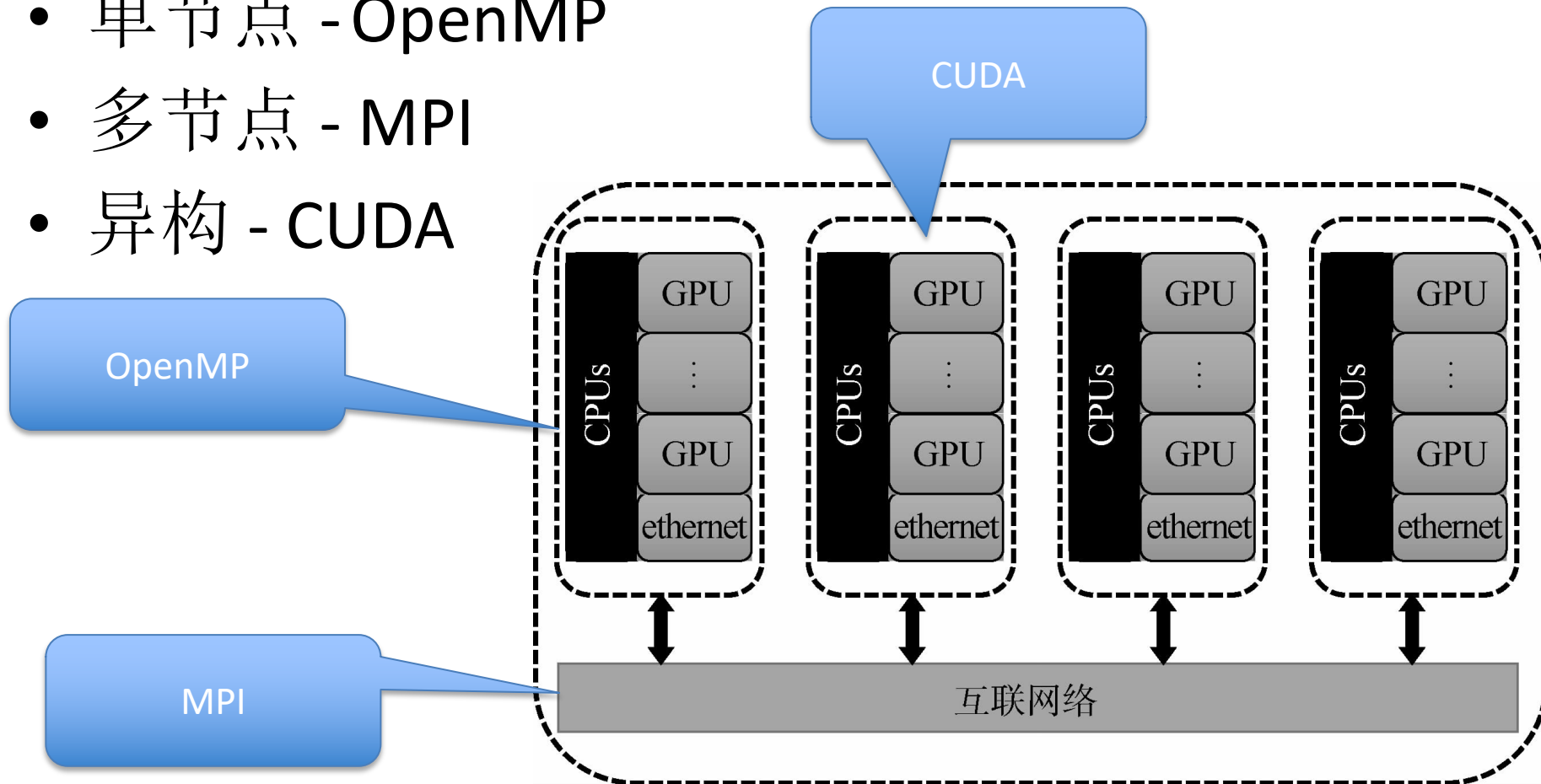
OpenMP基础

张超

中国人民大学信息学院

课程体系

- 单节点 - OpenMP
- 多节点 - MPI
- 异构 - CUDA



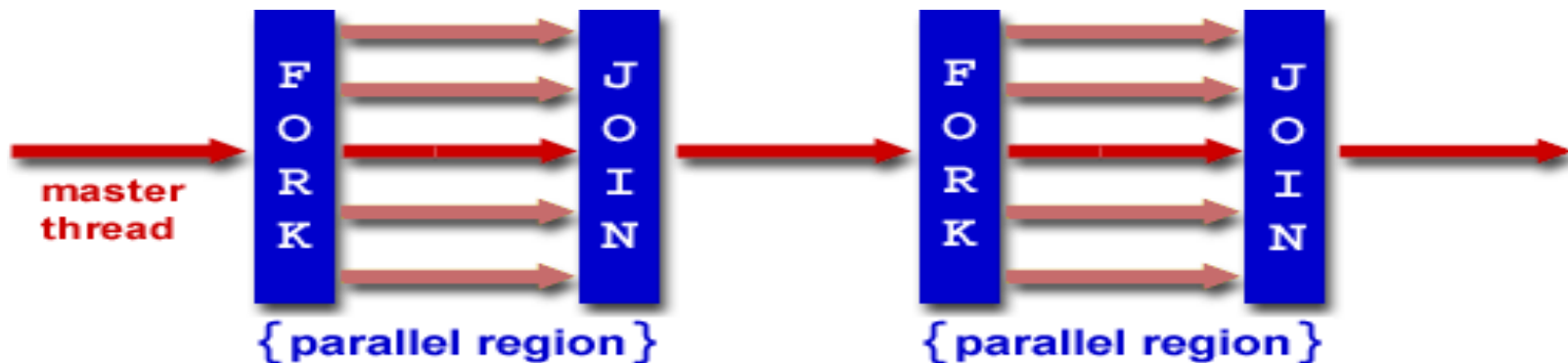


目录

- OpenMP简介
- OpenMP语法

OpenMP

- OpenMP是一套支持跨平台共享内存方式的多线程并发的编程API
- OpenMP (Open Multi-Processing) is a C/C++ and Fortran Application Programming Interface for **shared memory architectures**.
- OpenMP基于Fork、Join模型





OpenMP

历史版本

- 可以在[官网页面](#)查询到openmp的历史版本和发布日期

日期	版本
October 1997	Fortran 1.0
October 1998	C/C++ 1.0
March 2002	C/C++ 2.0
May 2005	OpenMP 2.5
May 2008	OpenMP 3.0
July 2011	OpenMP 3.1
July 2013	OpenMP 4.0
November 2015	OpenMP 4.5
November 2018	OpenMP 5.0
November 2020	OpenMP 5.1
November 2021	OpenMP 5.2



OpenMP与Pthread

OpenMP与Pthread有着许多本质的不同

- Pthread需要显式地明确每个线程的行为
- 只要系统拥有Pthreads库，Pthreads程序就能够被任意的C编译器使用
- Pthreads提供了虚拟地编写任何可知线程行为的能力
- OpenMP只需要简单的声明这块代码该并行执行
- 除了拥有OpenMP库，还要求编译器支持某些操作。
- OpenMP用于实现并行更加简单，但是很难对底层的线程交互进行编程。



OpenMP

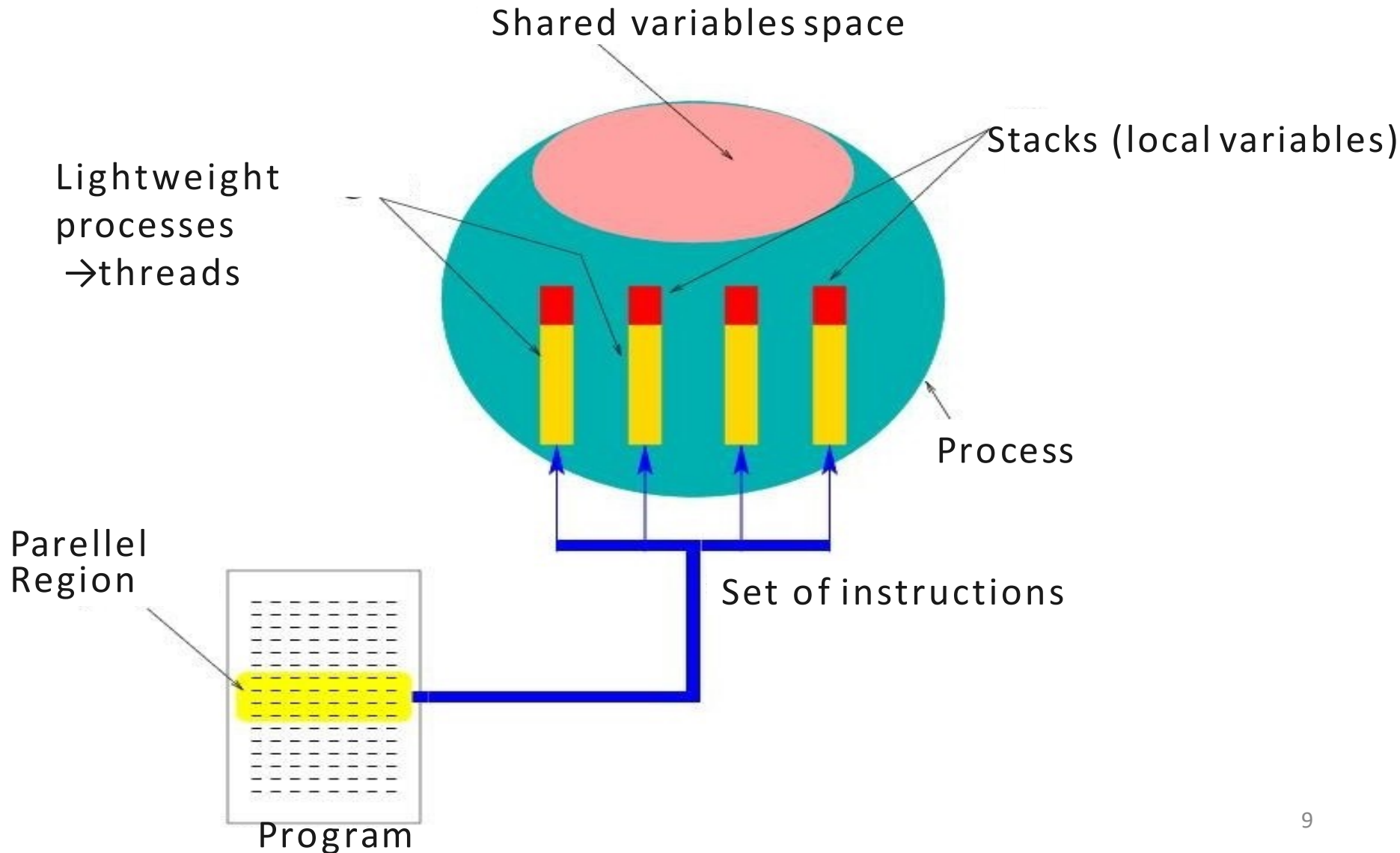
- OpenMP包括：
 - 一套编译指导语句
 - 函数调用库
 - 环境变量



OpenMP

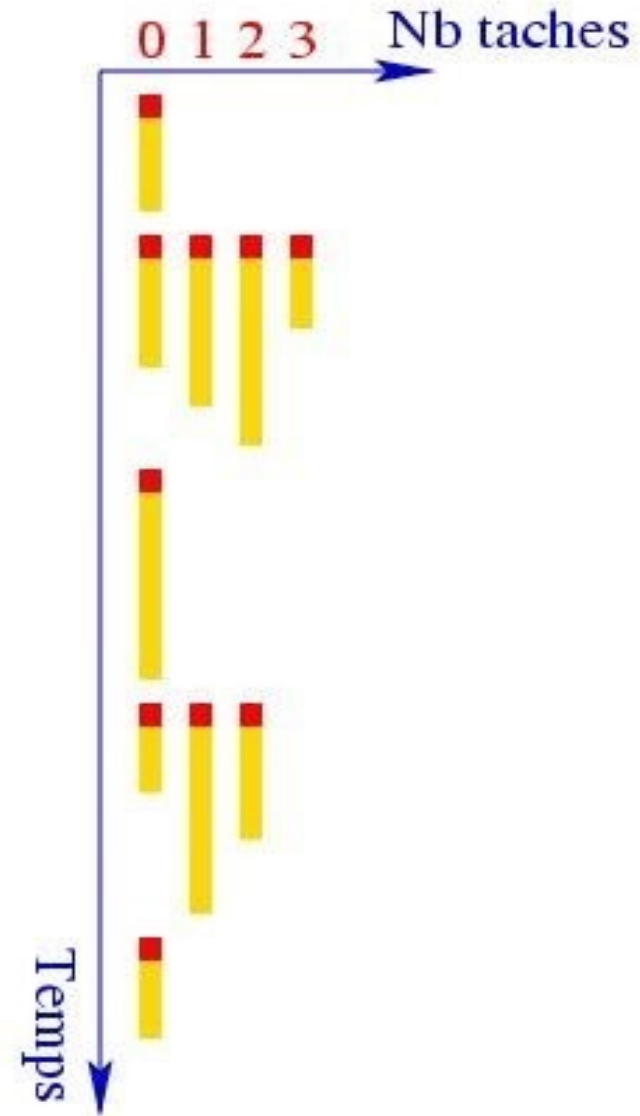
- 一个OpenMP程序由一个进程发起的
 - thread master
- 并行区域(parallel region)开始时, fork
 - 进程开启多个轻量级的线程
- 每个线程都执行并行区域中的代码
- 所有线程执行同样的代码
- 并行区域结束时(join), 只有主线程继续执行
- 在多个线程并行执行的过程中, 一个变量在如下情况可以read/write
 - 私有变量, 线程栈中
 - 共享变量, 共享内存空间中定义

OpenMP



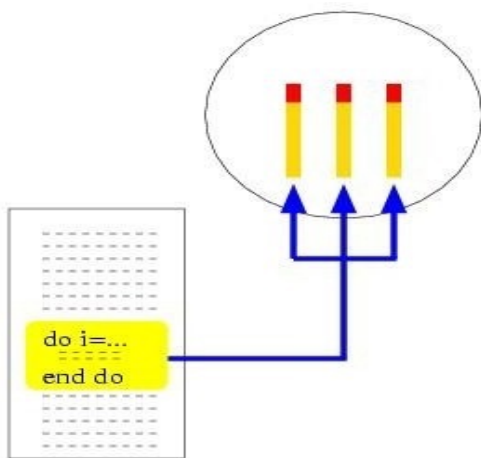
OpenMP

- OpenMP程序串行和并行区域交替出现
- 串行区域由master thread执行, thread 0
- 并行区域由多个线程同一时间一起执行
- 不同线程一起完成并行区域中的任务

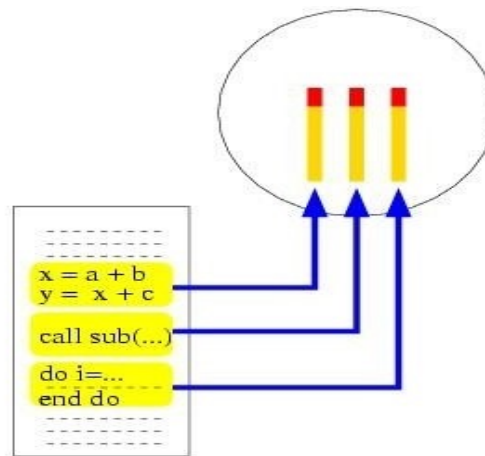


OpenMP

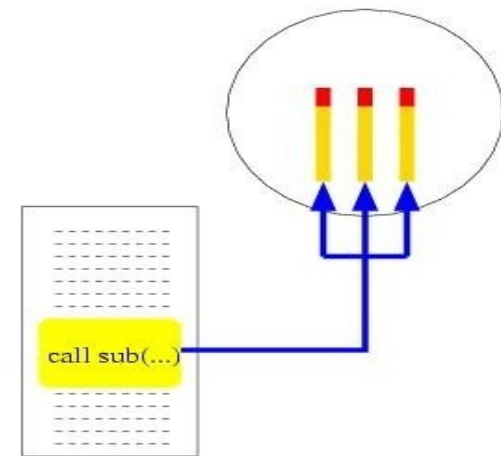
- OpenMP共同执行的任务
 - 不同的循环迭代
 - 不同的程序部分
 - 同样的程序部分



Loop Level
Parallelism



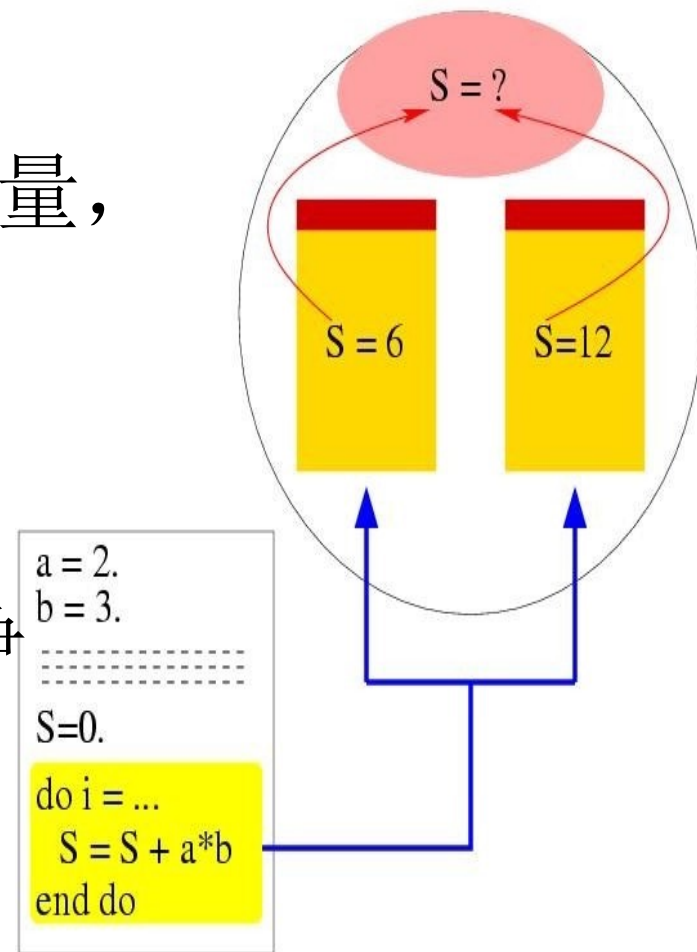
Parallel
Sections



Parallel
Procedures

OpenMP

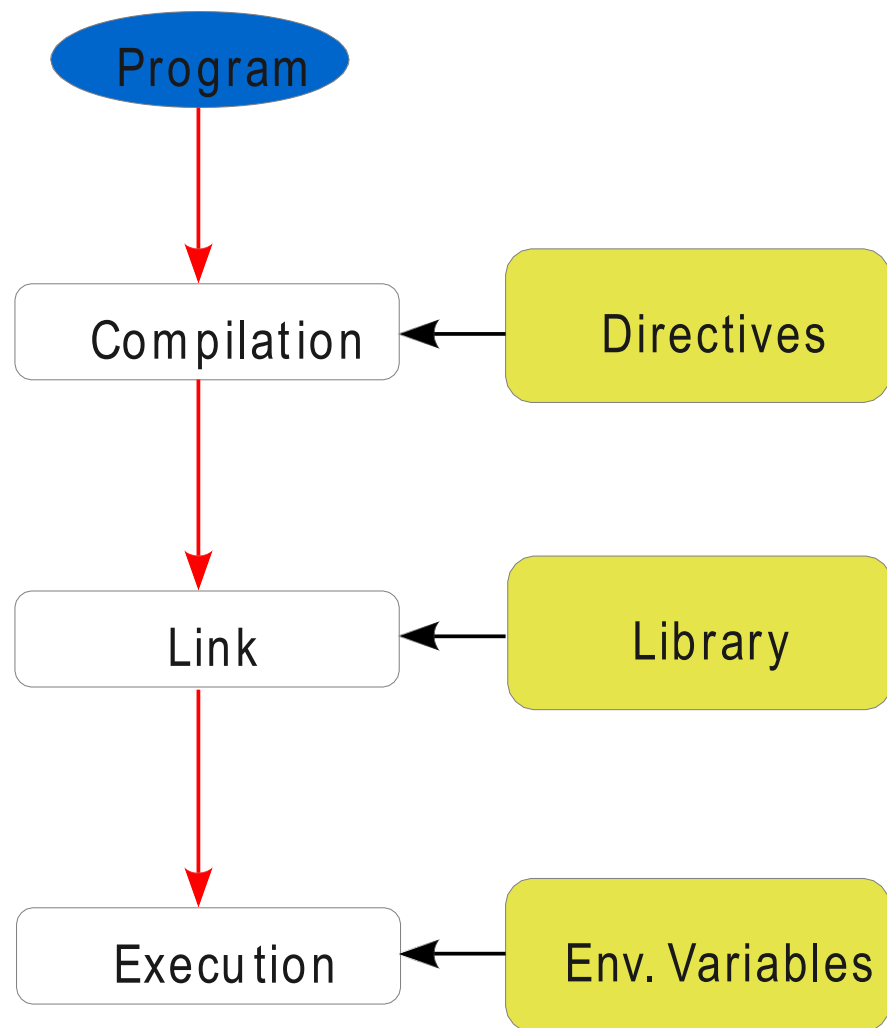
- 数据竞争data race
- 多个线程访问相同的共享变量，
只有一个写操作是有效的
- 通过线程间的同步
synchronization缓解数据竞争





OpenMP

- 编译指导语句：
 - 定义共同的任务
 - 同步
 - 变量的属性（私有 / 公有）
 - 如果编译指导语句不对，被当作一条注释
- 库函数调用，链接时装入
- 环境变量，设置好后执行时获得





OpenMP语法

- OpenMP指导语句：
 - 由程序员插入源代码中
 - 自动插入源代码中（自动并行）
- OpenMP指导语句格式：
#pragma omp directive [clause[clause]...] for C/C++
sentinel directive [clause[clause]...] for Fortran
- 程序中#include"omp.h":
- 定义了OpenMP函数，每个OpenMP程序都需要引用



OpenMP- Hello world

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void hello(void){
    int my_rank = omp_get_thread_num();
    int thread_count = omp_get_num_threads();
    printf("Hello from thread %d of %d.\n", my_rank, thread_count);
}

int main(int argc, char* argv[])
{
    int thread_count = strtol(argv[1], NULL, 10);

    # pragma omp parallel num_threads(thread_count)
    hello();

    return 0;
}
```



目录

- OpenMP
- OpenMP语法
 - Parallel Region
 - Work Sharing
 - Exclusive Execution
 - Synchronization
 - Nested Parallelism
 - API和环境变量
 - 常见错误
 - 更多内容



Parallel Region Construction

- 并行区域（parallel region）中默认变量是共享的
- 并行区域中所有线程执行同样的代码
- 并行区域结尾有显式的同步操作(barrier)
- 不可分支进入或离开并行区域

Parallel Region Construction

```
#include <stdio.h>
#include <omp.h>

int main(void)
{
    int a;        int p;
    a = 10000;    p = 0;
    #pragma omp parallel num_threads(8)
    {
        #ifdef _OPENMP
            p=omp_in_parallel();
        #endif
        printf("a = %d ; p = %d\n",a,p);
    }
    return 0;
}
```



Parallel Region Construction

```
> gcc ... -fopenmp -o omp omp.c  
> export OMP_NUM_THREADS=8  
> ./omp
```

```
a = 10000 ; p = 1  
a = 10000 ; p = 1  
a = 10000 ; p = 1  
a = 10000 ; p = 1  
a = 10000 ; p = 1  
a = 10000 ; p = 1  
a = 10000 ; p = 1  
a = 10000 ; p = 1
```



Parallel Region Construction

- 可以通过**DEFAULT**关键字来改变变量默认的状态。
- 如果变量的状态是***PRIVATE***，那么它会在线程的**stack**中。它的值在进入并行区域时是不确定的。



Parallel Region Construction

```
#include <stdio.h>
#include <omp.h>

int main(void)
{
    int a = 10000;
    int p = 0;
    #pragma omp parallel private(a)
    {
        #ifdef _OPENMP
            p=omp_in_parallel();
        #endif
        printf("a = %d ; p = %d\n",a,p);
    }
    return 0;
}
```



Parallel Region Construction

```
> gcc ... -fopenmp -o omp omp.c  
> export OMP_NUM_THREADS=8  
> ./omp
```

```
a = 0 ; p = 1  
a = 0 ; p = 1  
a = 0 ; p = 1  
a = 0 ; p = 1  
a = 32621 ; p = 1  
a = 0 ; p = 1  
a = 0 ; p = 1  
a = 0 ; p = 1
```



Parallel Region Construction

- 语句***FIRSTPRIVATE***会使私有变量初始化为并行区域开始时的数值



Parallel Region Construction

```
#include <stdio.h>
#include <omp.h>

int main(void)
{
    int a;
    a = 50000;

    #pragma omp parallel firstprivate(a)
    {
        a += 1111;
        printf("a = %d\n",a);
    }

    printf("After parallel region, a = %d\n",a);

    return 0;
}
```



Parallel Region Construction

```
> gcc ... fopenmp o omp omp.c  
> export OMP_NUM_THREADS=8  
> ./omp
```

```
a = 51111  
a = 51111  
a = 51111  
a = 51111  
a = 51111  
a = 51111  
a = 51111  
a = 51111
```

After parallel region, a = 50000



Scope of a Parallel Region

- 并行区域所影响的范围会包含区域内的子过程（subroutines） — ***the dynamic scope***



Scope of a Parallel Region

```
int main(void)
{
    void sub(void);

    #pragma omp parallel
    {
        sub();
    }
    return 0;
}
```

```
#include <stdio.h>
#include <omp.h>

void sub(void)
{
    int p=0;

    #ifdef _OPENMP
        p = omp_in_parallel();
    #endif

    printf("Parallel? :
           %d\n", p) ;
}
```



Scope of a Parallel Region

```
> gcc ... fopenmp o omp omp.c sub.c  
> export OMP_NUM_THREADS=8  
> ./omp
```

```
Parallel? : 1  
Parallel? : 1  
Parallel? : 1  
Parallel? : 1  
Parallel? : 1  
Parallel? : 1  
Parallel? : 1  
Parallel? : 1
```



Scope of a Parallel Region

- 在并行区域调用的子过程中，局部变量对每个线程是private，定义在各自的stack中

```
int main(void)
{
    void sub(void);

    #pragma omp parallel \
        default(shared)
    {
        sub();
    }
    return 0;
}
```

```
#include
<stdio.h>
#include
<omp.h>

void sub(void)
{
    int a;
    a = 50000;
    a = a +
    omp_get_thread_num();
    printf("a = %d\n", a);
}
```



Scope of a Parallel Region

```
> gcc ... -fopenmp o omp omp.c sub.c  
> export OMP_NUM_THREADS=8  
> ./omp
```

a = 50000

a = 50001

a = 50007

a = 50005

a = 50004

a = 50003

a = 50002

a = 50006



目录

- 并行体系结构
- 编程模型
- OpenMP
- OpenMP语法
 - Parallel Region
 - **Work Sharing**
 - Exclusive Execution
 - Synchronization
 - Nested Parallelism
 - API和环境变量
 - 常见错误
 - 更多内容



Work Sharing

- 使用OpenMP创建并行区域
- 程序员需要显式地对任务（**work**）和数据
进行划分，来保证没有数据竞争（**data race**）
- OpenMP有相应的指导语句：
 - FOR
 - SECTIONS
 - WORKSHARE



Work Sharing – FOR directive

编译器指令：

```
#pragma omp parallel for
```

它告诉编译器，接下来的for循环，将会使用并行的方式执行
使用并行的时候需要满足以下四个需求：

在循环的迭代器必须是可计算的并且在执行前就需要确定迭代的次数。

在循环的代码块中不能包含break, return, exit

在循环的代码块中不能使用goto跳出到循环外部。

迭代器只能够被for语句中的增量表达式所修改。



Work Sharing – FOR directive

- **for**用在并行区域中
- **for**中可以使用不同的语句：
- **private** : 使变量私有
- **firstprivate** : 使变量私有，同时将它赋值为**for**区域前最后一个数值
- **lastprivate** : 使变量私有，同时将它赋值为循环最后一次迭代的数值
- **reduction** : 每个线程有**private**的副本，**reduction**为共享变量在每个线程创建私有副本，变量会将操作作用在所有的私有共享变量（私有副本）上，最终结果写回全局共享变量。



Work Sharing – Reduction

Reduction(归并):

Reduction提供的操作符几乎都是符合结合律的二元操作符

$$\mathbf{a_1 \oplus a_2 \oplus a_3 \oplus \dots \oplus a_n}$$

本地变量的初始值如下所示:

Operator	Initial Value
+	0
*	1
-	0
^	0

Operator	Initial Value
&	~0
	0
&&	1
	0



Work Sharing – FOR directive

```
#define length 3

int main(void)
{
    int A[length];
    int B[length];
    int dot = 0;

    Init(A,B);

    #pragma omp parallel default(none) shared(A,B,dot)
    {
        int k;

        #pragma omp for reduction(+:dot)
        for (k=0 ; k<length ; ++k){
            dot += A[k] * B[k];
        }

    }

    return 0;
}
```

Work Sharing – FOR directive

```
#define length 3
```

```
int main(void)  
{
```

```
    int A[length];  
    int B[length];  
    int dot = 0;
```

```
    Init(A,B);
```

Length=3
A[3]={1,2,3}
B[3]={1,2,3}
Dot最后是多少?

```
#pragma omp parallel default(none) shared(A,B,dot)  
{
```

```
    int k;
```

```
    #pragma omp for reduction(+:dot)
```

```
    for (k=0 ; k<length ; ++k){
```

```
        dot += A[k] * B[k];
```

```
    }
```

```
}
```

```
    return 0;
```

```
}
```



提问2

```
#include<stdio.h>
#include<omp.h>
int main(){
    int a[3]={1,2,3};
    int b[3]={1,2,3};
    int dot=0;
```

```
#pragma omp parallel default(none) shared(a,b,dot)
```

```
{
```

```
    int k;
```

```
#pragma omp for reduction(+:dot)
```

```
    for(k=0; k<3; k++){
```

```
        printf("k=%d\n",k);
```

```
        dot+=a[k]*b[k];
```

```
    }
```

```
}
```

```
printf("dot=%d\n",dot);
```

```
return 0;
```

```
}
```

如果没有这条指导编译语句，假设8个线程，运行结果会是什么样？



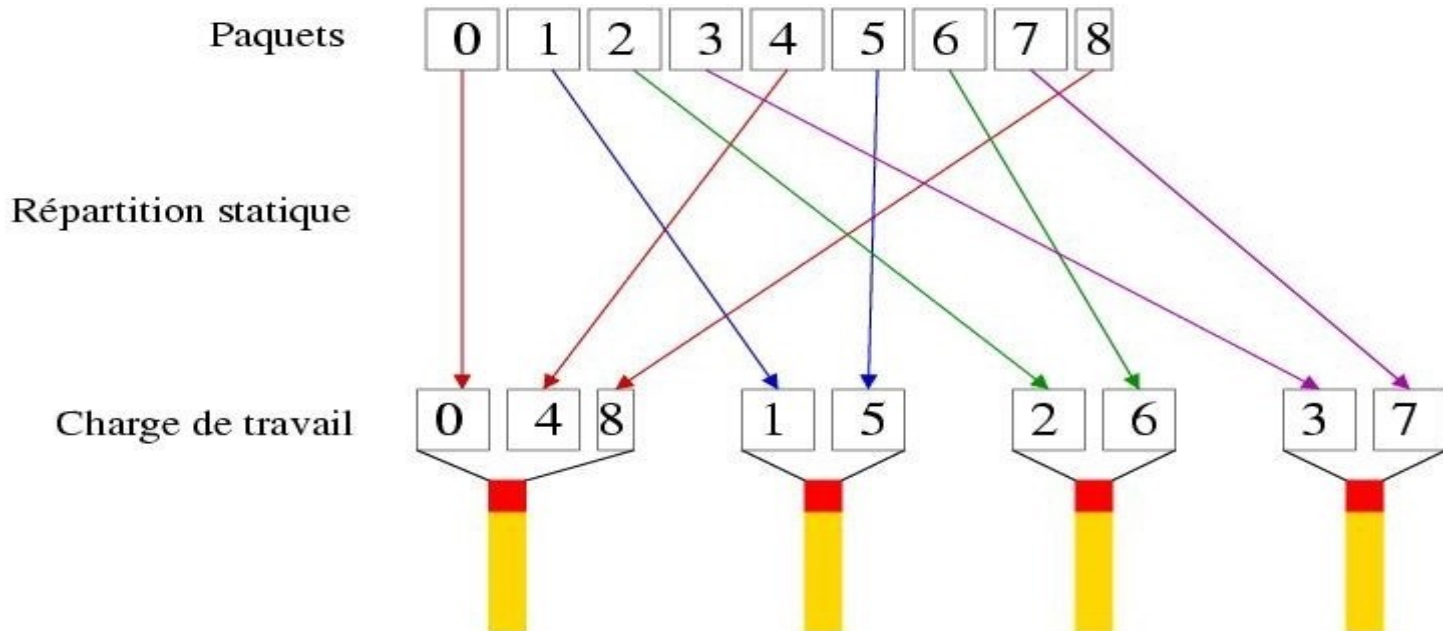
Work Sharing – FOR directive

- 线程间不同的调度策略
- static
- dynamic
- guided
- 语句SCHEDULE可选择不同的策略:
#pragma omp for schedule(static|dynamic|guided)
- 选择合适的调度策略对负载均衡很重要
- for语句后会有隐式的barrier，除非有语句
NOWAIT



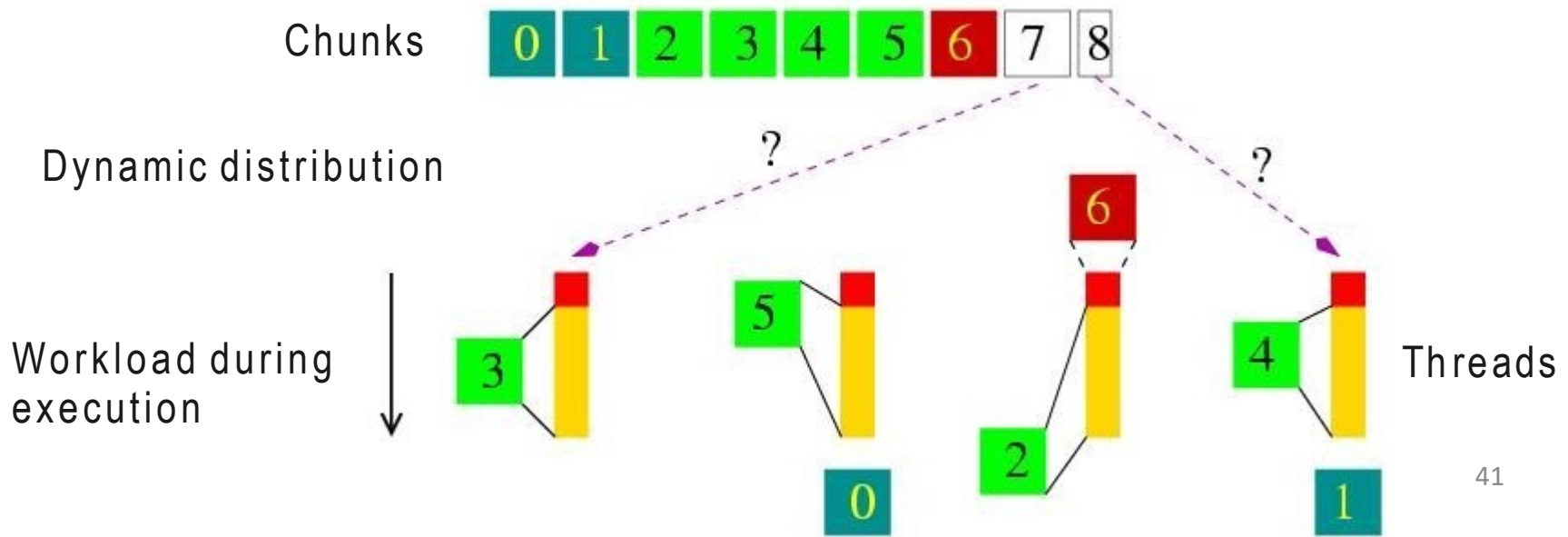
Work Sharing – FOR directive – SCHEDULE clause

- ***Schedule(static, chunk_size)*** 循环迭代以 *chunk_size* 分为不同的组 (chunk)，各组以循环的方式分配给不同的线程
- 如果 *chunk_size* 没有指定，则默认使各 chunk 尽可能均衡



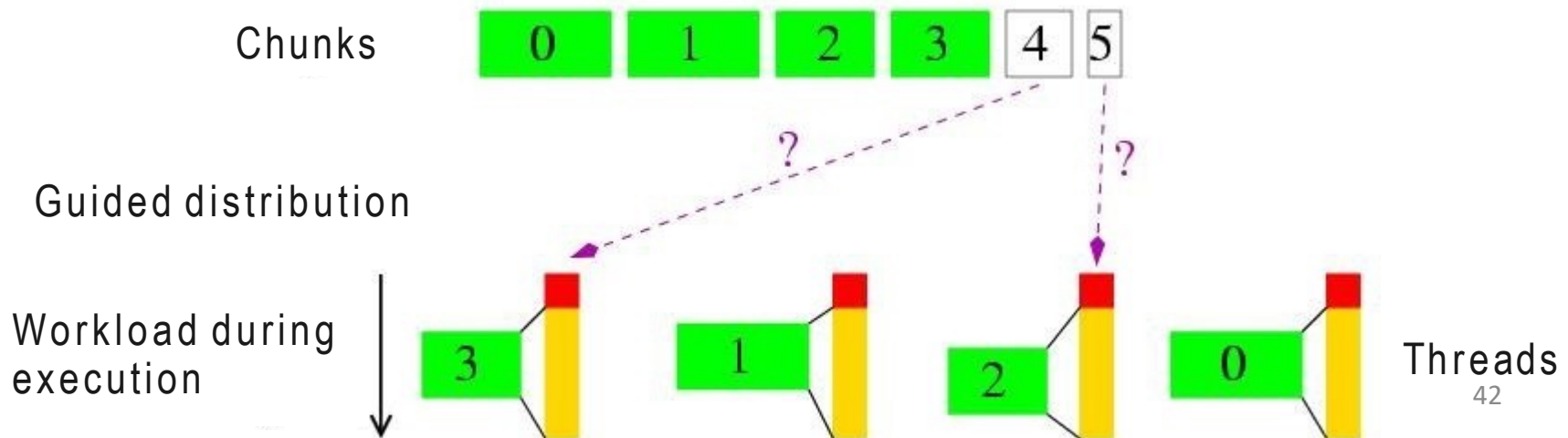
Work Sharing – FOR directive – SCHEDULE clause

- ***Schedule(dynamic, chunk_size)*** 循环迭代以 *chunk_size* 为单位分配给各线程。各线程完成对应的 chunk 后申请执行新的 chunk。
- 如果不指定，默认 *chunk_size* 是 1。



Work Sharing – FOR directive – SCHEDULE clause

- ***Schedule(guided, chunk_size)***调度是一种采用指导性的启发式自调度方法。开始时每个线程会分配到较大的迭代块，之后分配到的迭代块会逐渐递减。迭代块的大小会按指数级下降到指定的 *chunk_size* 大小。
- 如果不指定，默认 *chunk_size* 是1。



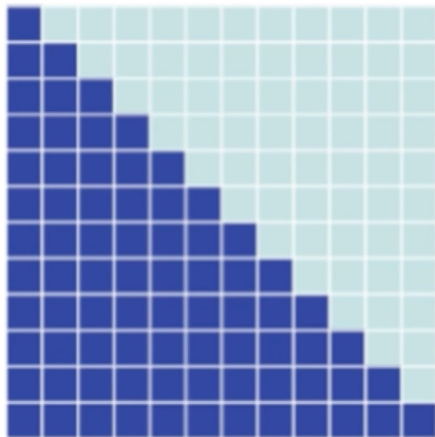


思考

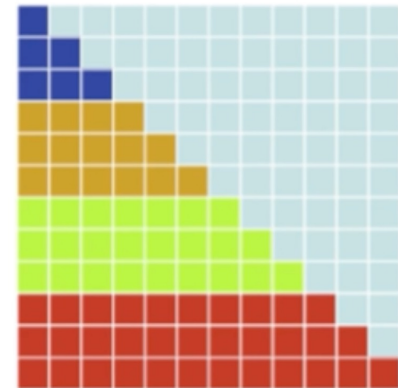
- static
- dynamic
- guided
- 几种策略的区别

Uniform Scheduling

```
for (int i = 0; i < 12; i++)  
    for (int j = 0; j <= i; j++)  
        a[i][j] = ...;
```



```
#pragma omp parallel for  
for (int i = 0; i < 12; i++)  
    for (int j = 0; j <= i; j++)  
        a[i][j] = ...;
```





Round-Robin Static Scheduling

Schedule(static[, chunk])

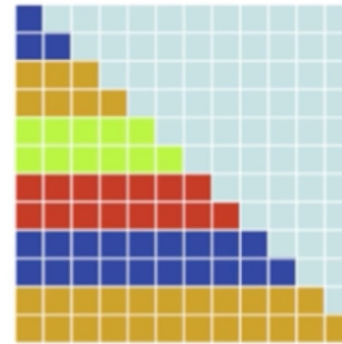
- 当采用static的参数时
- Chunk代表了每一块分块的大小
- 他会采取轮转制度，按照线程的数量，获得分配的一块大小的内容。在循环开始前就已经确定
- 低开销，但是可能会造成分配不均衡。

```
#pragma omp parallel for schedule(static, 2)
```

```
for (int i = 0; i < 12; i++)
```

```
for (int j = 0; j <= i; j++)
```

```
a[i][j] = ...;
```



Dynamic Scheduling

Schedule(dynamic[, chunk])

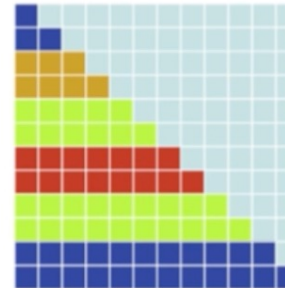
- 当采用dynamic的参数时
- Chunk代表了每一块分块的大小
- 每个线程执行完毕后，会自动获取下一个块。
- 高开销，但是能减少分配不均衡的情况。

```
#pragma omp parallel for schedule(dynamic, 2)
```

```
for (int i = 0; i < 12; i++)
```

```
    for (int j = 0; j <= i; j++)
```

```
        a[i][j] = ...;
```



Guided Scheduling

Schedule(guide[, chunk])

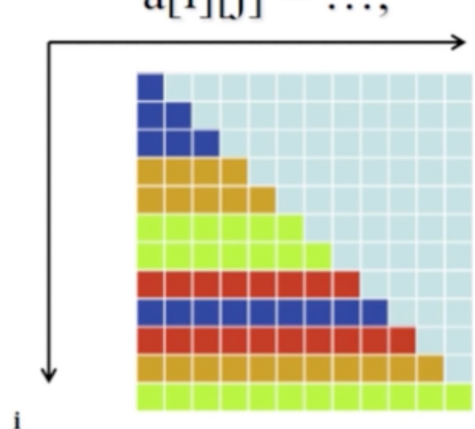
- 当采用guide的参数时，会按照一定的规则分配块
- 这是一种动态的分配。
- 每一块的分配的数量是在不断收缩的，但是最小不会小于chunk
- 最初的块会被定义成：
 - 循环数量/线程数
- 其余块的大小会被定义成：
 - 剩余循环数量/线程数

```
#pragma omp parallel for schedule(guided)
```

```
for (int i = 0; i < 12; i++)
```

```
for (int j = 0; j <= i; j++)
```

```
a[i][j] = ...;
```





目录

- 并行体系结构
- 编程模型
- OpenMP
- OpenMP语法
 - Parallel Region
 - Work Sharing
 - Exclusive Execution
 - Synchronization
 - Nested Parallelism
 - API和环境变量
 - 常见错误
 - 更多内容



Exclusive Execution

- 当程序员需要只使用一个线程执行任务时，可以使用如下语句使其他线程不参与执行
- SINGLE
- MASTER



Exclusive Execution – SINGLE directive

- SINGLE语句使相应的并行区域中只有一个线程在执行（不一定是master线程）
- 通常是第一个到达SINGLE语句的线程
- 除非有NOWAIT语句，其他的线程等待直到SINGLE语句对应的区域执行完成（implicit barrier）
- SINGLE语句创建时可以配合使用
 - private
 - firstprivate
 - copyprivate
- COPYPRIVATE语句将执行线程的private变量广播到其他线程



Exclusive Execution – SINGLE directive

```
#include <stdio.h>
#include <omp.h>

int main(void)
{
    int rank;
    int a;

    #pragma omp parallel private(a,rank)
    {
        a = 50000;

        #pragma omp single
        {
            a = -50000;
        }

        rank=omp_get_thread_num();
        printf("Rank : %d ; A = %d\n",
              rank,a);
    }
    return 0;
}
```

```
> gcc ...fopenmp o omp omp.c
> export OMP_NUM_THREADS=8
> ./omp
```

```
Rank : 6 ; A = 50000
Rank : 0 ; A = 50000
Rank : 3 ; A = 50000
Rank : 4 ; A = 50000
Rank : 1 ; A = 50000
Rank : 5 ; A = 50000
Rank : 7 ; A = -50000
Rank : 2 ; A = 50000
```



Exclusive Execution – SINGLE directive

```
#include <stdio.h>
#include <omp.h>

int main(void)
{
    int rank;
    int a;

    #pragma omp parallel private(a,rank)
    {
        a = 50000;

        #pragma omp single copyprivate(a)
        {
            a = -50000;
        }

        rank=omp_get_thread_num();
        printf("Rank : %d ; A = %d\n",
              rank,a);
    }
    return 0;
}
```

```
> gcc ...fopenmp o omp omp.c
> export OMP_NUM_THREADS=8
> ./omp
```

```
Rank : 6 ; A = -50000
Rank : 0 ; A = -50000
Rank : 3 ; A = -50000
Rank : 4 ; A = -50000
Rank : 1 ; A = -50000
Rank : 5 ; A = -50000
Rank : 7 ; A = -50000
Rank : 2 ; A = -50000
```



思考

- 为什么需要有 **Exclusive Execution**?

提问



目录

- 并行体系结构
- 编程模型
- OpenMP
- OpenMP语法
 - Parallel Region
 - Work Sharing
 - Exclusive Execution
 - Synchronization
 - Nested Parallelism
 - API和环境变量
 - 常见错误
 - 更多内容



Synchronization

- 同步很重要
- 保证所有线程都执行到了程序的同一指令
- 保证线程的执行顺序，对共享变量也有影响
- 用锁操作同步一些线程



Synchronization – Explicit BARRIER

- PARALLEL和FOR创建区域结束时都有隐式同步（barrier）； barrier可以通过NOWAIT语句绕过（bypass）
- 显式同步操作可以通过BARRIER语句声明，BARRIER指导语句同步同组的所有线程
- 每个线程执行到barrier后等待所有线程都到达才开始后续执行





Synchronization – Explicit BARRIER

```
#pragma omp parallel private(TID)
{
    double start, stop;

    TID=omp_get_thread_num();

    start = omp_get_wtime();

    if (TID < omp_get_num_threads()/2)
        sleep(3);

    stop = omp_get_wtime();

    printf("%.0f seconds elapsed before barrier, thread %d\n",
           stop-start, TID);

    #pragma omp barrier

    stop = omp_get_wtime();

    printf("%.0f seconds elapsed after barrier, thread %d\n", stop
           -start, TID);
}
```



Synchronization – Explicit BARRIER

```
> gcc ... fopenmp.o omp omp.c  
> export OMP_NUM_THREADS=8  
> ./omp
```

```
0 seconds elapsed before barrier, thread 7  
0 seconds elapsed before barrier, thread 5  
0 seconds elapsed before barrier, thread 4  
0 seconds elapsed before barrier, thread 6  
3 seconds elapsed before barrier, thread 0  
3 seconds elapsed before barrier, thread 3  
3 seconds elapsed before barrier, thread 1  
3 seconds elapsed before barrier, thread 2  
3 seconds elapsed after barrier, thread 1  
3 seconds elapsed after barrier, thread 0  
3 seconds elapsed after barrier, thread 7  
3 seconds elapsed after barrier, thread 2  
3 seconds elapsed after barrier, thread 4  
3 seconds elapsed after barrier, thread 3  
3 seconds elapsed after barrier, thread 6  
3 seconds elapsed after barrier, thread 5
```

Synchronization – ATOMIC directive



- **ATOMIC**语句能够保证内存中的共享存储位置在某时刻只被同一线程修改
- 多个线程执行同一语句对某位置同时读、写会产生不确定的数值
- **ATOMIC**语句应用在声明后所创建的区域中



Synchronization – ATOMIC directive

```
#include <stdio.h>
#include <omp.h>
```

```
int main(void)
{
    int counter, rank;

    counter = 50000;

    #pragma omp parallel default(none) \
        private(rank) shared(counter)
    {
        rank=omp_get_thread_num();

        #pragma omp atomic
        counter++;

        printf("Rank : %d ; counter = %d\n",
               rank, counter);
    }

    printf("In total, counter = %d\n", counter);
    return 0;
}
```

```
> gcc ... fopenmp.o omp omp.c
> export OMP_NUM_THREADS=8
> ./omp
```

```
Rank : 0 ; counter = 50002
Rank : 7 ; counter = 50001
Rank : 1 ; counter = 50003
Rank : 3 ; counter = 50004
Rank : 4 ; counter = 50005
Rank : 2 ; counter = 50006
Rank : 6 ; counter = 50007
Rank : 5 ; counter = 50008
In total, counter = 50008
```

Synchronization – CRITICAL directive



- **CRITICAL**语句保证程序的特定区域一次（时刻）只有一个线程执行
- **CRITICAL**语句可以看成是更通用的**ATOMIC**语句
- 为了更好的性能，建议不要用**CRITICAL**语句替换**ATOMIC**语句



CRITICAL与ATOMIC的区别

```
#pragma omp parallel for
{
    for ( I = 0; I < n; i++){
        #pragma omp critical
        x[index[i]] += workOne(i)
        y[index[i]] += workTwo(i)
    }
}
```

Critical 保护内容:

- workOne()的调用
- Index[i]的寻找
- x[index[i]] 与workOne(i)的加法运算
- 对x数组的赋值

最终对于x的更新来说, 这是串行执行的

```
#pragma omp parallel for
{
    for ( I = 0; I < n; i++){
        #pragma omp atomic
        x[index[i]] += workOne(i)
        y[index[i]] += workTwo(i)
    }
}
```

atomic 保护内容:

- x[index[i]] 与workOne(i)的加法运算
- 对x数组的赋值

同样不会产生更新的错误

相对于critical来说, 只有index的访问是不安全的, 其他都是安全的。

Synchronization – CRITICAL directive

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
int main(void)
```

```
{
```

```
    int s, p;
```

```
    s = 0, p = 1;
```

```
    #pragma omp parallel
```

```
{
```

```
        #pragma omp critical
```

```
{
```

```
            s++;
```

```
            p*=2;
```

```
        }
```

```
    }
```

```
    printf("Total sum and product: %d, %d\n", s, p);
```

```
    return 0;
```

```
}
```

Export OMP_NUM_THREADS=8

S=?

P=?



目录

- 并行体系结构
- 编程模型
- OpenMP
- OpenMP语法
 - Parallel Region
 - Work Sharing
 - Exclusive Execution
 - Synchronization
 - **Nested Parallelism**
 - API和环境变量
 - 常见错误
 - 更多内容



嵌套并行

Nested Parallelism

- OpenMP允许嵌套并行
- 嵌套并行指在一个并行区域中嵌套另一个并行区域
- 注意：线程ID是对每个并行区域局部的（local），可能会有不同线程具有同一个ID
- 优点：不同层间利用并行性
- 缺点：并行区域创建、销毁开销



Nested Parallelism

```
#pragma omp parallel
{
    #pragma omp for
    for(i=0; i<n ; ++i) {
        ...
    }

    #pragma omp parallel
    {
        work(...);
    }
}
```

```
void work(...) {
    /* declarations */
    #pragma omp for
    for (j=0; j<m; ++j)
    {
        ...
    }
}
```



Nested Parallelism

- `#include <omp.h>`
- `#include <stdio.h>`
- `#include <time.h>`
- `#include <unistd.h>`

```
void fun() {  
    #pragma omp parallel for  
    for (int i = 0; i < 2; ++i) {  
        #pragma omp parallel for  
        for (int j = 0; j < 2; ++j) {  
            printf("%d %d\n", i, j);  
            sleep(1); } } }  
  
int main() {  
    omp_set_nested(1);  
    fun();  
    omp_set_nested(0);  
    fun();  
    return 0; }
```



目录

- 并行体系结构
- 编程模型
- OpenMP
- OpenMP语法
 - Parallel Region
 - Work Sharing
 - Exclusive Execution
 - Synchronization
 - Nested Parallelism
 - API和环境变量
 - 常见错误
 - 更多内容



程序接口

- ***omp_set_num_threads()***
 - 设定并行区域中的线程数目
- ***omp_get_num_threads()***
 - 返回并行区域中的线程数目
- ***omp_get_thread_num()***
 - 返回目前组中的当前线程的ID
- ***omp_get_max_threads()***
 - 返回并行区域中可以创建的最大线程数目



程序接口

- ***omp_get_num_procs()***
 - 返回逻辑处理器数目
- ***omp_in_parallel()***
 - 判断是否在并行区域中
- ***omp_set_nested()***
 - 设置并行区域嵌套
- ***omp_get_nested()***
 - 判断是否允许嵌套



环境变量

- ***OMP_NUM_THREADS***
 - 并行区域中最大的线程数目
- ***OMP_SCHEDULE = static/dynamic/guided [chunk_size]***
 - 当使用the ***schedule(runtime)*** 语句时选择的调度策略
- ***OMP_NESTED***
 - 是否允许OpenMP嵌套并行



目录

- 并行体系结构
- 编程模型
- OpenMP
- OpenMP语法
 - Parallel Region
 - Work Sharing
 - Exclusive Execution
 - Synchronization
 - Nested Parallelism
 - API和环境变量
 - 常见错误
 - 更多内容



常见错误

- 私有变量
- **Private**类型在进入并行区域时没有定义数值
- 出并行区域时没有定义如何保留数值

```
int n = 10;           // shared
int a = 7;            // shared

#pragma omp parallel for
for (int i = 0; i < n; i++) // i private
{
    int b = a + i;      // b private
    ...
}
```

常见错误

Private Variables

```
int main(int argc, char **argv)
{
```

```
    int i,a,b,c,n;
```

```
    n=atoi(argv[1]);
    a = b = 0;
```

n=4

最后c为多少？

```
    #pragma omp parallel for private(i,a,b)
```

```
    for (i=0; i<n; ++i) {
```

```
        ++b;
```

```
        a = b + i;
```

```
    }
```

```
    c = a + b;
```

```
    return c;
```

```
}
```

常见错误

Private Variables

```
int main(int argc, char **argv)
{
    int i,a,b,c,n;

    n=atoi(argv[1]);
    b = 0;

    #pragma omp parallel for firstprivate(b)\
                             lastprivate(a,b)

    for (i=0; i<n; ++i) {
        ++b;
        a = b + i;
    }
    c = a + b;
    return c;
}
```

n=4

最后c为多少？



常见错误

- MASTER创建
- 忘记MASTER创建区后面没有隐式的barrier

```
int main(void)
{
    int xInit, xLocal;

    #pragma omp parallel shared(xInit) private(xLocal)
    {
        #pragma omp master
        { xInit = 10; }

        xLocal = xInit;
    }
}
```



常见错误

- 总结
 - 使用Use **default(*none*)**
 - 检查并行区域中变量的范围
 - 初始化私有变量

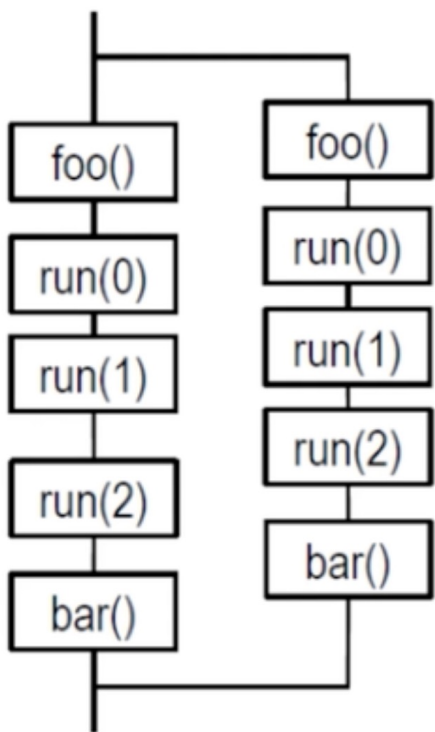


目录

- 并行体系结构
- 编程模型
- OpenMP
- OpenMP语法
 - Parallel Region
 - Work Sharing
 - Exclusive Execution
 - Synchronization
 - Nested Parallelism
 - API和环境变量
 - 常见错误
 - 更多内容

SECTIONS语句

Parallel语句使用的时候，接下来的一段代码块都将会以并行的形式执行

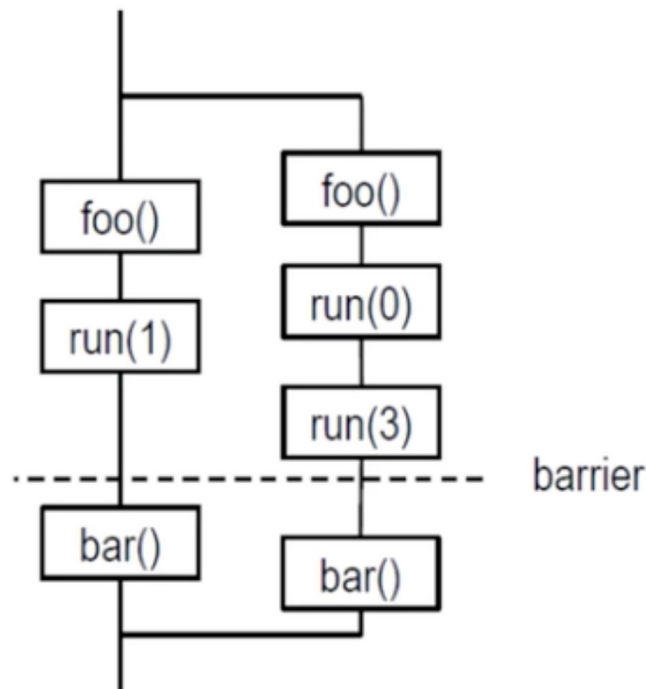


```
#pragma omp parallel \  
num_threads(2)  
{  
    foo();  
    for (int i = 0; i < 3; i++)  
        run(i);  
    bar();  
}
```

SECTIONS语句

在使用了parallel之后，我们可以在代码块中进行一系列的操作。

```
#pragma omp parallel \  
num_threads(2)  
{  
    foo();  
    #pragma omp for  
    for (int i = 0; i < 3; i++)  
        run(i);  
    bar();  
}
```





SECTIONS语句

- `section`指由一个线程执行的代码区域段
- `SECTIONS`区域内可以创建多个`block`，每个`block`可以由一个线程执行
- 多个块间是独立的
- 不同的块用`SECTION`语句在`SECTIONS`定义的区域内容义
- 如果一个线程够快，该线程可能执行多个`section`



SECTIONS语句

- 所有的SECTION语句应该定义在SECTIONS语句范围中
- 除非用NOWAIT， SECTIONS后会有隐式的barrier
- SECTIONS中可用的语句有：
 - private
 - firstprivate
 - lastprivate
 - reduction



SECTIONS语句

例子:

```
#pragma omp parallel shared(a,b,c,d) private(i)
{
    #pragma omp sections
    {
        #pragma omp section
        {
            for (i=0; i<N; i++)
                c[i] = a[i] + b[i];
        }
        #pragma omp section
        {
            for (i=0; i<N; i++)
                d[i] = a[i] * b[i];
        }
    } /* end of sections */
} /* end of parallel section */
```

SECTION语句



```
#include <stdio.h>
#include <omp.h>

int main()
{
    int    rank;

    #pragma omp parallel private(rank)
    {
        rank=omp_get_thread_num();
        #pragma omp sections nowait
        {
            #pragma omp section
            {
                printf("[Thread %d] \n",
rank);
            }
            #pragma omp section
            {
                printf("[Thread %d] \n",
rank);
            }
        }
    }
    return 0;
}
```

```
> gcc ... -fopenmp o omp omp.c
> export OMP_NUM_THREADS=8
> ./omp
```

```
[Thread 7]
[Thread 0]
```



SECTIONS语句

```
int main(int argc, char* argv[])
{
    int thread_count = strtol(argv[1], NULL, 10);
    int j = 10;
    printf("j = %d\n", j);
    # pragma omp parallel num_threads(thread_count) reduction(+:j)
    {
        # pragma omp sections
        {
            # pragma omp section
            {
                int my_rank = omp_get_thread_num();
                int thread_count = omp_get_num_threads();
                j = j + 10;
                printf("Hello from thread %d of %d in section 1 and j = %d.\n", my_rank, thread_count, j);
            }
            # pragma omp section
            {
                int my_rank = omp_get_thread_num();
                int thread_count = omp_get_num_threads();
                j = j + 20;
                printf("Hello from thread %d of %d in section 2 and j = %d.\n", my_rank, thread_count, j);
            }
        }
    }
    printf("j = %d\n", j);

    return 0;
}
```



MASTER语句

- MASTER语句创建的区域由master线程执行（线程0）
- MASTER区域结尾没有隐式同步



```
#include <stdio.h>
#include <omp.h>

int main()
{
    int rank;
    int a;

    #pragma omp parallel
    private(a,rank)
    {
        a = 50000;

        #pragma omp master
        {
            a = -50000;
        }

        rank=omp_get_thread_num();
        printf("Rank : %d ; A =
%d\n",rank,a);
    }
    return 0;
}
```

```
> gcc ... fopenmp o omp omp.c
> export OMP_NUM_THREADS=8
> ./omp
```

```
Rank : 6 ; A = 50000
Rank : 0 ; A = -50000
Rank : 3 ; A = 50000
Rank : 4 ; A = 50000
Rank : 1 ; A = 50000
Rank : 5 ; A = 50000
Rank : 7 ; A = 50000
Rank : 2 ; A = 50000
```



数据依赖

- OpenMP编译器不检查被parallel for指令并行化的循环所包含的迭代间的依赖关系
- 一个或者更多个迭代结果依赖于其他迭代的循环，一般不能被正确的并行化



数据依赖

数据依赖性:

在该循环中的计算依赖于一个或更多个先前的迭代结果。例如, 考虑下述代码, 计算前n个斐波那契数:

```
fibo[0] = fibo[1] = 1;  
#pragma omp parallel for num_threads(thread_count)  
for(int i = 2; i < 10; i++)  
    fibo[i] = fibo[i-1]+fibo[i-2];
```

谢谢！



中國人民大學
RENMIN UNIVERSITY OF CHINA